JKU
LINZ INSTITUTE
OF TECHNOLOGY

Recurrence CFD for 3D Lid-Driven Cavity



from CAS setup to Successful rCFD Prediction

Md Zahid Hasan
Artificial Intelligence

2026-06-01

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Spin-up CFD Simulation



Outline

- Physical model and governing equations
- Why a dedicated start-up run?
- Kinetic energy monitoring
- `CFD_start.scm` script workflow
- From transient to stationary regime
- Interpretation for experts

3D Lid-Driven Cavity
Nitrogen, Re based on lid velocity

Goal

Generate a fully developed, statistically stationary velocity field as baseline for all subsequent scalar CFD and rCFD simulations.

Governing Equations & Physical Model

Fluid: Nitrogen (N_2), single-phase, Newtonian, constant properties

- Density ρ = 1.7 $\mathrm{kg/m^3}$
- Dynamic viscosity μ = 1.76 $\mathrm{kg/(m \cdot s)}$

Unsteady incompressible Navier-Stokes with turbulence model

$$\nabla \cdot \mathbf{u} = 0$$

$$\rho \frac{\partial \mathbf{u}}{\partial t} + \rho(\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nabla \cdot \left[\mu_{\text{eff}} \left(\nabla \mathbf{u} + \nabla \mathbf{u}^T\right)\right]$$

where $\mu_{\text{eff}} = \mu + \mu_t$ (molecular + eddy viscosity). No body forces.

Numerical method
Pressure-velocity coupling (SIMPLE/PISO/coupled) and spatial discretisation

Why a Dedicated Start-up Run?

- Initial condition (e.g. quiescent fluid) contains strong transients
- Early spin-up of main vortex, secondary eddies, turbulence field adjustment
- Contaminates both recurrence database and long-time mixing metrics

Two main purposes:

1. Generate converged, physically meaningful base flow → _START case
2. Provide kinetic energy history $E_{kin}(t)$ to identify statistically stationary regime

Without start-up
Transient contamination
↓
Unreliable rCFD database

With start-up
Stationary attractor
↓
Solid baseline for all later simulations

Domain-integrated kinetic energy Monitoring

$$E_{\text{kin}}(t) = \frac{1}{2} \int_{\Omega} \rho \ |\mathbf{u}(\mathbf{x}, t)|^2 \ \mathrm{d}V$$

- UDF `DEFINE_EXECUTE_AT_END(CFD_write_energy)` loops over all fluid cells
- Computes local kinetic energy density, integrates over Ω
- Appends pair (t, E_{kin}) to `CFD_start_energy.out`

Stationary regime criterion

When $E_{\text{kin}}(t)$ transitions from monotonic drift to fluctuations around a constant level → system has reached the statistically stationary attractor.

`CFD_start.scm` – Global Parameters (1/2)

Parameter	Value / Purpose
`ANSYS_Fluent_dir`	Folder with initial case and destination for `_START` case
`ANSYS_Fluent_file`	`LidDrivenCavity_3D`
`CFD_user_src_dir`	Directory containing UDF source `CFD_user.c` and plotting script
`number_of_CFD_episodes`	600
`simulation_timestep_width`	0.01 s
`timesteps_per_CFD_episode`	10

Total simulated physical time:

$$600 \times 10 \times 0.01\ \text{s} = 60\ \text{s}$$

CFD_start.scm – Global Parameters (2/2)

The _INIT case contains: geometry, mesh, nitrogen material, moving lid BC, and turbulence model settings.

Note

The _INIT case is the starting point; time integration begins from this state.

Script Blocks – Workflow Overview

CFD_1: Prepare working directory

- Copy `_INIT.cas.h5`, `CFD_user.c`, Python script

CFD_2: Compile UDF

- Remove old library, build `libudf_cfd`

CFD_3: Read initial case & initialize

- Load case, attach UDF library, standard flow initialization

CFD_5: Time integration with monitoring

- Attach `CFD_write_energy` as execute-at-end hook
- Loop over 600 episodes (10 inner iterations per time step, Δt = 0.01 s)

CFD_6: Write START case & generate plot

- Save final state as `LidDrivenCavity_3D_START.cas.h5`
- Call Python to plot $E_{\text{kin}}(t)$

CFD_7: Archive & clean up

Time Integration – CFD_5 in Focus (1/2)

Dual-time stepping

For each outer time step $\Delta t = 0.01$ s, perform 10 inner iterations to converge pressure-velocity coupling and turbulence equations.

UDF hook:

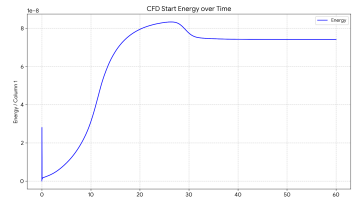
```
/define/user-defined/function-hooks/execute-at-end "CFD_write_energy
```

- Called after *every* outer time step
- Computes and logs $E_{\text{kin}}(t)$

Time Integration – CFD_5 in Focus (2/2)

Expected behaviour

Initially $E_{\text{kin}}(t)$ rises as the moving lid accelerates the fluid, passes through a transient overshoot, then approaches a plateau where lid work balances dissipation.



Kinetic energy evolution during start-up (schematic)

Archiving and Outputs (CFD_6 & CFD_7)

After completion of 60 s physical time:

- Final state written to `LidDrivenCavity_3D_START.cas.h5`
- `_START` case contains fully developed velocity, pressure, and turbulence fields
- It becomes the reference state for:
  - `CFD_run.scm` (scalar CFD reference runs)
  - `rCFD_prep.scm` (recurrence CFD database preparation)
  - `rCFD_run_topright_3D.scm` (rCFD acceleration)

Clean-up:

- Move `*.out`, `*.trn`, images to `post\CFD_start`
- Move `*.h5` files back to `.`
  `ansys_fluent` and Remove temporary files and logs

Interpretation

Key points for the specialist

- The start-up simulation is a **deliberate solution of the unsteady incompressible Navier-Stokes system** – not an arbitrary warm-up.
- The execute-at-end UDF provides a **quantitative measure** of the system's approach to the attractor via $E_{\mathrm{kin}}(t)$.
- When the energy signal has ceased to drift and fluctuates around a statistically steady value do we commit the _START case.

Baseline is essential for:

- Building a clean recurrence database
- Long-time scalar transport runs without transient artefacts

Conclusion & Next Steps

Achieved:

- Robust, physically meaningful baseline flow
- Automated spin-up via `CFD_start.scm`
- Verified stationary regime via $E_{kin}(t)$ monitoring
- Ready-to-use _START case on disk

Next steps in the tutorial:

1. Scalar CFD reference runs (passive scalar transport)
2. **rCFD database preparation (Next)**
3. Recurrence CFD acceleration (3D lid-driven cavity)

Take-home message

"A clean start-up guarantees trustworthy rCFD and scalar transport results."

Tutorial for Preparing the ANSYS CAS File

Access the complete tutorial for setting up your CFD case files:

Click Here to Access the Tutorial Drive Folder

Includes:

- Step-by-step guide for `.cas` file preparation
- Model definitions and boundary conditions

rCFD Database Preparation



Conceptual Role of rCFD Preparation

Goal: Shift from flow spin-up to *learning* the recurrence structure and local transport properties of the converged field.

- **Data Generator:** Still utilizes full CFD solver loops but acts purely to track patterns along the attractor.
- **Mesh Mapping:** Re-loads `_START` files aligning Fluent's mesh partitioning directly with rCFD data arrays.
- **Instrumentation:** Introduces 9 User-Defined Scalars (UDS) coupled to local turbulence metrics.
- **Sampling Run:** 5-second monitored trajectory captures local effective diffusivity boundaries and velocity pairs.

Core Deliverable

The execution of `rCFD_write_Rec` evaluates the recorded database to output the recurrence matrix driving accelerated passive scalar transport.

Mathematical Principles: Attractor Sampling The database preparation phase samples the Navier–Stokes attractor to construct a clean recurrence matrix (R_{ij}) and an accompanying stitching path.

The Recurrence Norm:

$$R_{ij} = \|\mathbf{u}_i - \mathbf{u}_j\|_{L^2}$$

- Measures the cell-wise L^2 velocity field distance between snapshots i and j.
- Evaluated throughout the 203 monitoring episodes (sampled every 5 solver steps).

Stitching Path Mechanics

Using the compiled database, the library identifies off-diagonal minima (valleys of recurrence) to find recurring cycles. It chooses a pathway that jumps between snapshots with minimal R_{ij} to loop across the attractor seamlessly.

Mathematical Principles: Transport Calibration Simultaneously, diagnostic scalar fields characterize transient domain mixing via an augmented transport equation:

$$\frac{\partial(\rho c_i)}{\partial t} + \nabla \cdot (\rho \mathbf{u} c_i) = \nabla \cdot \left(\Gamma_{\text{eff}} \nabla c_i\right) + S_{c_i}$$

- **Diffusivity Matching:** Diagnostic field diffusivity is coupled directly to the flow's turbulent viscosity scales.
- **Local Corrections:** Captures fine-grained spatial mixing details to map out Γ_{eff}.

Purpose of Calibration

This information calibrates the face-swap model and local diffusivity corrections used later to offset geometric interpolation discrepancies when skipping velocity fields in rCFD.

Prep Scripts: Environment Setup & Compilation

rCFD_p1: Workspace Assembly

```
(define (rCFD_p1)
  !copy LidDrivenCavity_3D_START.cas.h5 .
  !copy LidDrivenCavity_3D_START.dat.dat.h5 .
  !copy ..\..\src\*.c .
  !copy ..\..\src\*.h .
  !copy ..\..\third-party\tomlc17\*.c .
  ; Sets up compilation control flags...
)
```

- Pulls the fully converged baseline dataset into a clean operational directory.
- Copies core source codes and TOML parameter parsers.

rCFD_p2: Compiling UDF Engines

```
(define (rCFD_p2)
  !if exist libudf_rcfd_prep rmdir /s /q ...
  !move .\rCFD_user_prep.h .\rCFD_user.h
  (load "..\..\src\rCFD_C2C_prep_compile.scm")
  !del *.c
  !del *.h
)
```

- Purges old test architectures.
- Contextually links user configurations (`rCFD_user_prep.h`) into the general engine compilation tree.

Prep Scripts: Memory Allocation & Topology

rCFD_p3: Initializing Data Struct Mappings

```
(define (rCFD_p3)
  /file/read-case .\LidDrivenCavity_3D_START.cas.h5
  /define/user-defined/user-defined-memory 10 q
  /define/user-defined/user-defined-node-memory 2 q
  /define/user-defined/compiled-functions load "libudf_rcfd_prep"
  /file/read-data .\LidDrivenCavity_3D_START.dat.h5
  /define/user-defined/execute-on-demand "rCFD_init_all::libudf_rcfd_prep"
  /define/user-defined/execute-on-demand "rCFD_write_Topo::libudf_rcfd_prep"
)
```

- Allocates User-Defined Memory (UDMI) lines for structural data logging.
- `rCFD_init_all`: Binds Fluent cell coordinates to internal rCFD bookkeeping arrays.

Prep Scripts: Instrumentation Fields

rCFD_p4a: Allocating Diagnostic Diffusion Fields

```
(define (rCFD_p4a)
  /define/user-defined/user-defined-scalars 9 ...
  /define/materials/change-create nitrogen nitrogen yes defined-per-uds
    0 expression "TurbulenceViscosity" ... 8 expression "TurbulenceViscosity"
  /solve/set/discretization-scheme/ uds-0 1 ... uds-8 1
  /define/boundary-conditions/set/wall 5 6 7 () uds-bc ... "rCFD_wall_bc::libudf_rcfd_prep"
  /file/write-case-data .\LidDrivenCavity_3D_START_rCFD_PREP.cas.h5
)
```

- Spawns 9 passive test fields. These are diagnostic indicators, distinct from your actual mixing variables.
- Ties diffusion rates directly to turbulence metrics via specific wall UDF conditions (`rCFD_wall_bc`).

Prep Scripts: Attractor Tracking
rCFD_p5: Automated Time Integration Sequence

```
(define (rCFD_p5)
  /define/user-defined/function-hooks/execute-at-end
    "CFD_write_gas_vol::libudf_cfd" "rCFD_execute_at_end::libudf_rcfd_prep" ""
  (do ((i 0 (+ i 1))) ((= i number_of_monitoring_episodes)) ; 203 episodes
    /solve/set/time-step simulation_timestep_width          ; dt = 0.005 s
    /solve/dual-time-iterate monitoring_timesteps_per_episode 30) ; 5 inner steps
)
```

- Simulates a tightly checked physical time window (203 × 5 × 0.005 s ≈ 5 s).
- rCFD_execute_at_end activates every 5 iterations to pull current vector and scalar profiles directly into the internal tracker.

Prep Scripts: Path Matrix Generation & Cleanup

rCFD_p7: Compiling the Recurrence Matrix

```
(define (rCFD_p7)
  /define/user-defined/execute-on-demand
    "rCFD_write_Rec::libudf_rcfd_prep"
)
```

- Launches post-processing data analysis to map distance metrics and output structural trajectory chains.

rCFD_p8: Deallocating Active Memory Arrays

```
(define (rCFD_p8)
  /define/user-defined/execute-on-demand
    "rCFD_free_all::libudf_rcfd_prep"
)
```

rCFD_p9: Data Organization

- Relocates target `.h5` maps and data history lists back to primary project directories.
- Purges log files and code fragments.

Pipeline Wrapper

```
(define (rCFD_prep)
  (rCFD_p1) (rCFD_p2) (rCFD_p3)
  (rCFD_p4a) (rCFD_p5) (rCFD_p6)
  (rCFD_p7) (rCFD_p8) (rCFD_p9)
)
```

Summary: rCFD Database Preparation

- **Instrumented Run:** Built directly over your converged _START state to extract systemic signature maps.
- **Automated Engine Tracking:** Leverages specialized dynamic hooks and Scheme sub-flows to record accurate velocity snapshots and localized mixing fields.
- **Mathematical Baseline:** Establishes the structural similarity matrices and stitching sequences needed to skip fluid calculations safely during accelerated transport.

Take-home Message

"The preparation phase captures the flow's physical and transient signature, enabling massive runtime reductions down the line."

rCFD Prediction & Industrial Deployment



rCFD Prediction Phase: Input Map Architecture

The script `rCFD_run_topright_3D.scm` automates the prediction phase by locking your analysis execution to frozen parameters:

1. Converged Flow Field Baseline

- `rCFD_r1` sets up a workspace.
- `rCFD_r3` reads original N–S solution (`_START.cas.h5` / `.dat.h5`).

2. Frozen Recurrence Path Data

- Calls `rCFD_init_all` for mappings.
- `rCFD_read_DB` pulls prep-phase data chains directly into RAM.

3. Scenario User Headers

- Swaps definitions: `rCFD_user_run_topright.h` → `rCFD_user.h`.
- `rCFD_C2C_run_compile.scm` builds localized solver engine (`libudf_rcfd_run`).

Scenario Configuration: Patch Geometry & Channels The file `rCFD_user_run_topright.h` defines what substance properties are simulated, where transport starts, and how fast particles smooth out.

Circular Patch Initial Placement:

- Domain boundaries: $0.05 \times 0.05 \times 0.005\ \mathrm{m}$.
- Cylindrical shape in top-right sector: Centre $(C_X, C_Y) = (0.035, 0.035)\ \mathrm{m}$, Radius $C_R = 0.005\ \mathrm{m}$, full depth layer $z \le 0.005\ \mathrm{m}$.
- Inside the boundary: `user_init_data` forces concentrations $c_{\mathrm{MIX1}} = c_{\mathrm{MIX2}} = 1.0$ (twin red blobs).

Material Transport Channels:

- **Carrier fluid:** Nitrogen ($\rho = 1.7\ \mathrm{kg/m^3}$, $\mu = 1.76 \times 10^{-5}\ \mathrm{kg/(m \cdot s)}$).
- **Target scalars:** Dual concentration indicators (MIX1, MIX2) tracked simultaneously.

Scenario Configuration: Transport Settings & Outputs

rCFD Internal Transport Settings:

- Time-step width Δt = 0.025 s.
- 5 steps per frame over 800 monitoring loops (25 s of active timeline).

Face-Swap Diffusivity Values:

- **MIX1:** `face_swap_diffusion` = 0.00002 (Low value → slow, distinct smoothing).
- **MIX2:** `face_swap_diffusion` = 0.03 (High value → rapid scalar smoothing).

Calculated Output Metrics (user_post):

- **Mixing index:** Global mass-weighted deviation curve tracking scalar decay over time.
- **Center of Gravity (CoG):** Coordinates (x, y, z) tracking positional paths.
- Outputs logged into `.out` files and copied back to Fluent UDMI 0/1.

Prediction Metrics: What the Run Produces When calling `(rCFD_run)`, `rCFD_r5_post` triggers 800 lightweight episodes that push the field along the recorded recurrence track:

Mixing Index Homogenization

- Curves approach 1.0 (fully mixed state) over 20–25 s.
- MIX2 rises faster, verifying that larger face-swap corrections accelerate smoothing while retaining macro-flow physics.

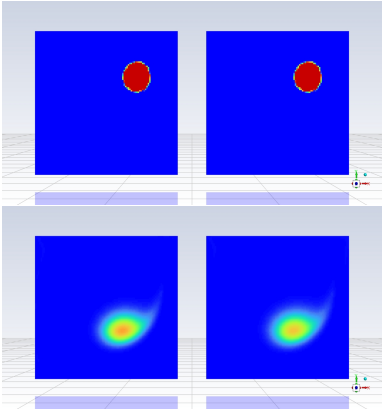
CoG Cyclical Trajectories

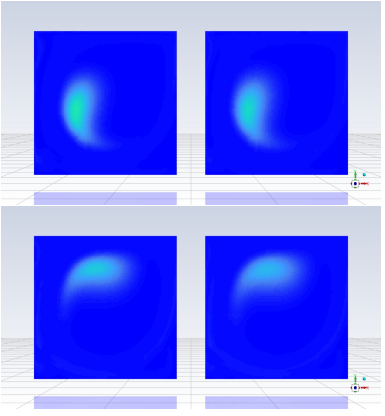
- Nearly closed loops in the (x, y)-plane reflect the periodic rotational vortices.

Visual Field Observation

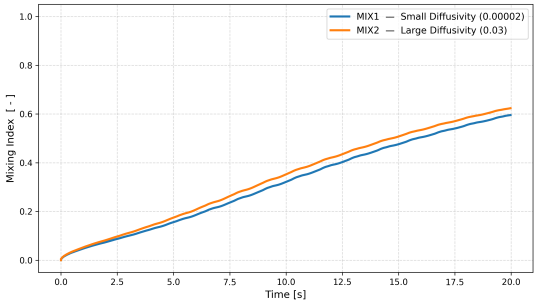
Contour images output side-by-side: the left block displays MIX1 (retaining sharp localized fluid gradients) while the right block displays MIX2 (showing fast structural dissipation). Both are driven identically by the same background vortex.

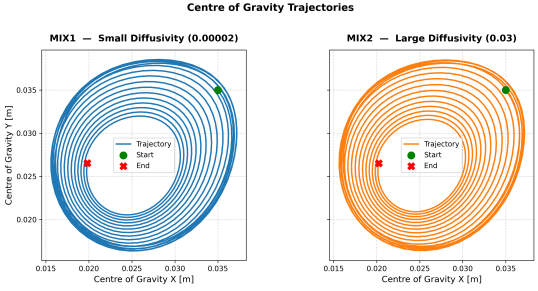
Recorded Contours During rCFD Run





Result from UDF





Parametric Design Optimization: Rapid What-If Studies Because rCFD decouples background fluid dynamics from passive mass transport equations, the framework generalizes flawlessly across varied scenarios.

Fixed Core Assets (Compute Once)

- Background grid architecture.
- Base Navier–Stokes velocity patterns.
- Attractor similarity paths and indexes.

Variable Parameters (Predict Many)

- Injection spots (center vs wall, multiple sources).
- Patch shapes (complex mask outlines, volumetric bands).
- Additive properties (molecular variations, turbulent scales).

Speed Advantage

Altering parameters requires updating only `user_init_data` or `Data_Dict` and re-running the scalar transport equations. This runs in seconds or minutes.

Industrial Performance Advantage

- **One-Time Capital Investment:** The heavy, high-fidelity CFD solver execution and database preparation are completed exactly once per operating condition.
- **Lightweight Design Iteration:** Complex configuration queries are resolved instantly with lightweight, scalable transport calculations.
- **Resource Optimization:** Reduces computational cost by orders of magnitude while preserving large-scale flow physics.

Industrial Take-home Message

"By decoupling frozen flow dynamics from scalar variables, rCFD transforms full Navier–Stokes attractors into instant, reusable playgrounds for rapid transport optimization."

Files for rCFD

Preparation of files for the Prediction

watch this tutorial



LINZ INSTITUTE
OF TECHNOLOGY